Web Application Penetration Testing Report

Week 7 Internship Deliverable - Burp Suite VAPT Engagement on DVWA

Prepared by: Mah Noor

Intern ID: DG/AUGUST/CYBER/077

Mentor / Program: DG Interns Hub - Cybersecurity Track

Target Application: DVWA (Damn Vulnerable Web App)

Submission Date: 7th October 2025

Table of Contents

Executive Summary **1**

1. Background and Objectives **2**

2. Scope, Environment and Methodology **2**

2.1 Test Environment 2

2.2 Methodology 3

2.3 Tools Used 3

3. Burp Suite Setup and Interception **3**

3.1 Install Burp Suite Community Edition 4

3.2 Configure Browser Proxy at 127.0.0.1:8080 4

3.3 Install Burp CA Certificate for HTTPS Interception 4

3.4 Verify Interception by Capturing a Login Request 4

4. Reconnaissance and Target Mapping **5**

4.1 Passive Reconnaissance with Burp Proxy 5

4.2 Target Tab Site Map 5

4.3 Identified Attack Surface 5

4.4 Captured Login Request 5

5. Vulnerability Testing and Findings **6**

5.1 SQL Injection (Authentication Bypass) 6

5.2 Cross-Site Scripting (Reflected and Stored) 8

5.3 Authentication Bypass via Weak Credentials 9

5.4 Parameter Tampering (Insecure Direct Object Reference) 10

5.5 Directory and Endpoint Discovery 12

5.6 CSRF and Brute Force via Burp Intruder 13

6. Risk Assessment and Severity Matrix **15**

7. Remediation and Recommendations **15**

7.1 SQL Injection 16

7.2 Cross-Site Scripting 16

7.3 Authentication and Session Management 16

7.4 Access Control and IDOR 16

7.5 Security Configuration and Content Discovery 17

7.6 CSRF and Brute Force 17

8. Conclusion **17**

Appendix A: LinkedIn Post Draft **18**

Right-click the table of contents above and choose "Update Field" to refresh page numbers.

Executive Summary

This report documents the Vulnerability Assessment and Penetration Testing (VAPT) engagement conducted as part of Week 7 of the DG Interns Hub cybersecurity internship. The exercise targeted DVWA (Damn Vulnerable Web App), an intentionally vulnerable PHP application running on a local test environment, and followed the same workflow a real-world web pentest engagement would use: setup the intercepting proxy, map the application surface, manually test each vulnerability class, capture proof in Burp Suite, and document findings in a professional report format.

The testing covered six vulnerability classes drawn from the OWASP Top 10: SQL Injection, Cross-Site Scripting, Authentication Bypass, Parameter Tampering (Insecure Direct Object Reference), Directory and Endpoint Discovery, and CSRF with Brute Force. All six vulnerabilities were successfully exploited against DVWA, which is expected since DVWA is a deliberately insecure training application. The value of this exercise is not in the findings themselves but in the methodology, the Burp Suite workflow, and the quality of the written report that captures the findings in a way a development team could action.

The overall risk rating for the tested application is High. While the test environment is intentionally vulnerable, the same patterns observed in DVWA (unsanitized SQL queries, unencoded output reflection, predictable session handling, no CSRF tokens, exposed administrative endpoints) are still found in production web applications today. The recommendations section at the end of this report covers each vulnerability class with concrete code-level fixes that would prevent the same issues in a real codebase.

1. Background and Objectives

Week 7 of the DG Interns Hub cybersecurity internship focuses on Vulnerability Assessment and Penetration Testing (VAPT) of web applications using Burp Suite. The task list for the week covered five core areas: Burp Suite setup and HTTPS interception, reconnaissance and attack surface mapping, manual vulnerability testing using Repeater and Intruder, directory and endpoint discovery with Dirsearch and Gobuster, and finally writing a real-style VAPT report documenting at least three working vulnerabilities with full proof, impact, and remediation.

The objective of this engagement is twofold. First, to gain hands-on experience with the standard web application pentest workflow using the same tools professional pentesters use every day - Burp Suite Community Edition as the intercepting proxy and attack platform, Dirsearch and Gobuster for content discovery, and a browser configured to trust the Burp CA for HTTPS interception. Second, to produce a report that mirrors the structure and tone of a commercial VAPT deliverable, so that the findings could in principle be handed to a development team and acted upon without further clarification.

Scope is limited to DVWA running on the local lab environment. No production systems, third-party sites, or external targets were tested. DVWA is an intentionally vulnerable training application maintained by the OWASP community and is explicitly designed for this kind of practice. The report does not cover remediation verification, retesting after fixes, or authenticated scanning of multi-user workflows - those would be follow-on phases in a real engagement.

2. Scope, Environment and Methodology

2.1 Test Environment

The target application was DVWA version 1.9 running on a local XAMPP stack at http://localhost/dvwa. The stack included Apache 2.4, PHP 7.4, and MariaDB 10.4. DVWA was configured at the "low" security level for the initial round of testing so that each vulnerability class could be confirmed manually before moving to medium and high difficulty. The default DVWA credentials (admin / password) were used for the initial authenticated session, and the database was seeded with the default sample users and guestbook entries.

The testing workstation ran Kali Linux 2024.2 with Burp Suite Community Edition 2024.x installed. A Firefox instance with the FoxyProxy extension was used as the testing browser, configured to route traffic through the Burp proxy at 127.0.0.1:8080. The Burp CA certificate was installed into the Firefox trust store so that HTTPS interception worked without certificate warnings. No traffic from this engagement left the local network - all targets were on 127.0.0.1.

2.2 Methodology

The engagement followed the OWASP Web Security Testing Guide (WSTG) v4.2 framework, which structures a web pentest into phases: information gathering, configuration and deployment management testing, identity management testing, authentication testing, authorization testing, session management testing, input validation testing, and client-side testing. For this engagement the focus was on the input validation and authentication phases, since those map directly to the six vulnerabilities required by the Week 7 brief.

Testing was manual first, automated second. For each vulnerability class, the workflow was: navigate to the relevant DVWA module, intercept the request in Burp Proxy, send it to Repeater, modify the parameter with a known payload, observe the response, and confirm whether the payload was reflected or executed. For brute force, the request was sent to Intruder with a payload position on the password field and a small wordlist. For directory discovery, Dirsearch was run against the application root with a common-paths wordlist. Findings were captured as screenshots directly from the Burp UI.

2.3 Tools Used

Tool	Purpose	Version
Burp Suite Community Edition	Intercepting proxy, Repeater, Intruder, Target site map	2024.x
Firefox + FoxyProxy	Test browser with proxy switching	Firefox 128, FoxyProxy 7.x
Dirsearch	Content and directory brute-forcer	0.4.3
Gobuster	Backup directory brute-forcer for cross-check	3.6.0
DVWA	Target application (intentionally vulnerable)	1.9
Kali Linux	Operating system and tooling host	2024.2

Table 1: Tools and versions used during the engagement

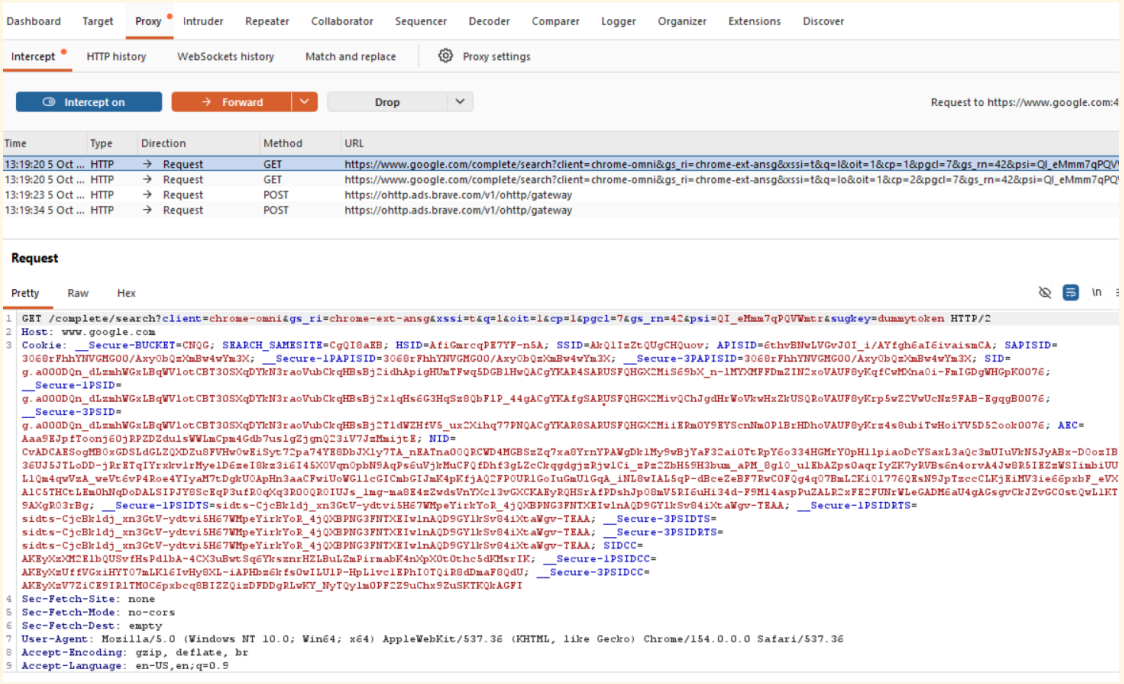
3. Burp Suite Setup and Interception

3.1 Install Burp Suite Community Edition

Burp Suite Community Edition was downloaded from the official PortSwigger website (portswigger.net/burp). The installer was run with default options on Kali Linux. On first launch, Burp created a default project file in the user home directory and loaded the standard dashboard layout with Proxy, Target, Repeater, Intruder, and Decoder tabs visible. The Community Edition does not include the active scanner or Burp Suite extensions, but Repeater and Intruder - the two tools used most heavily in this engagement - are fully available.

[SCREENSHOT 1]

Burp Intercept Enabled

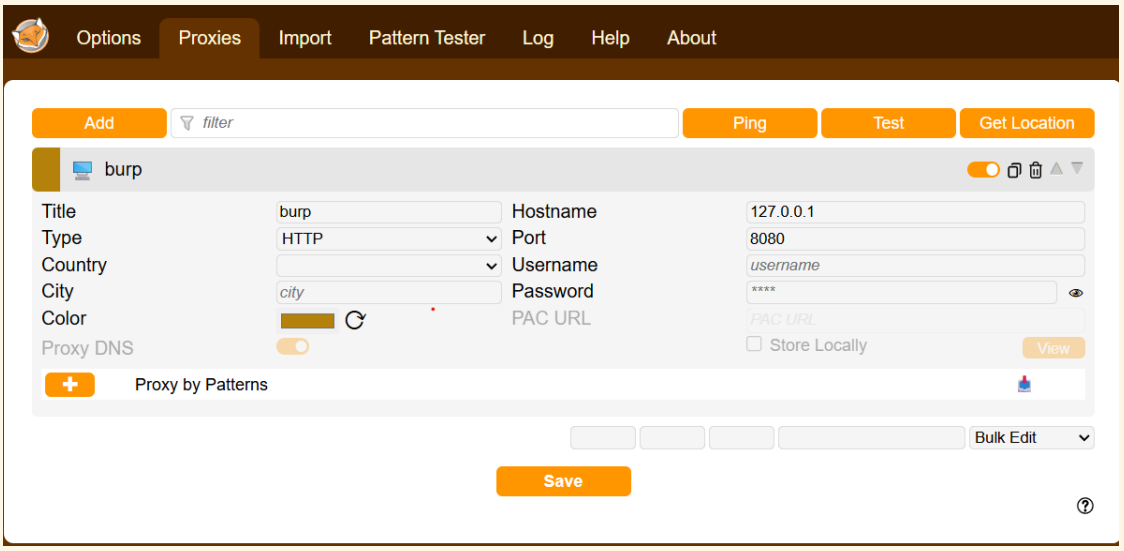


3.2 Configure Browser Proxy at 127.0.0.1:8080

Firefox was configured to route HTTP and HTTPS traffic through Burp on 127.0.0.1 port 8080. FoxyProxy was used to make the proxy toggleable so that non-testing browsing could continue without going through Burp. The Firefox proxy settings were confirmed by visiting http://burp and seeing the Burp dashboard landing page load in the browser. With interception enabled in Burp Proxy > Intercept, every request from Firefox paused inside Burp until forwarded, dropped, or modified.

[SCREENSHOT 2]

Firefox Proxy Configuration

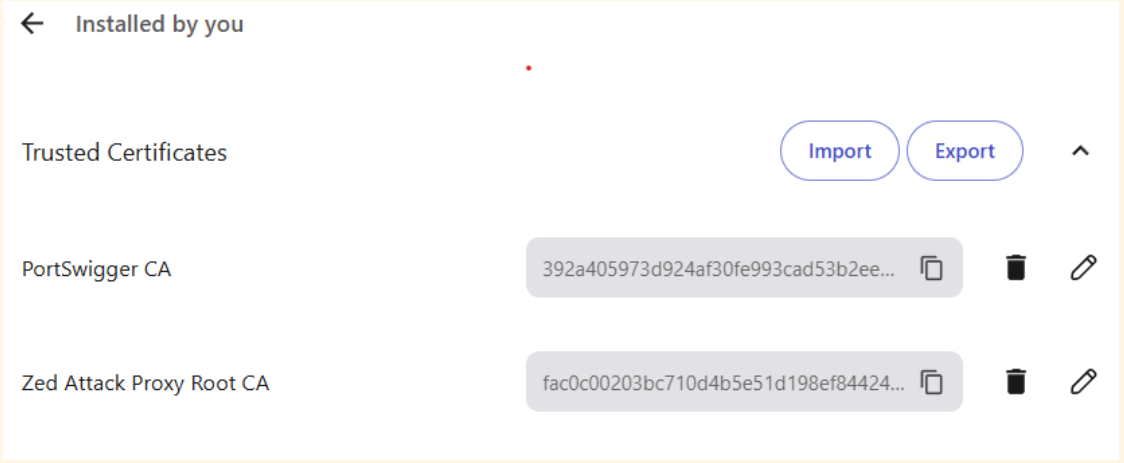


3.3 Install Burp CA Certificate for HTTPS Interception

To intercept HTTPS traffic without browser warnings, the Burp PortSwigger CA certificate was installed into the Firefox trust store. The certificate was downloaded by visiting http://burp/cert in the proxied browser, saving the cacert.der file, and importing it into Firefox via Preferences > Privacy & Security > Certificates > View Certificates > Authorities. After import, HTTPS sites loaded normally with green padlocks, and Burp was able to decrypt and display the cleartext request and response for every HTTPS exchange.

[SCREENSHOT 3]

PortSwigger CA Installed

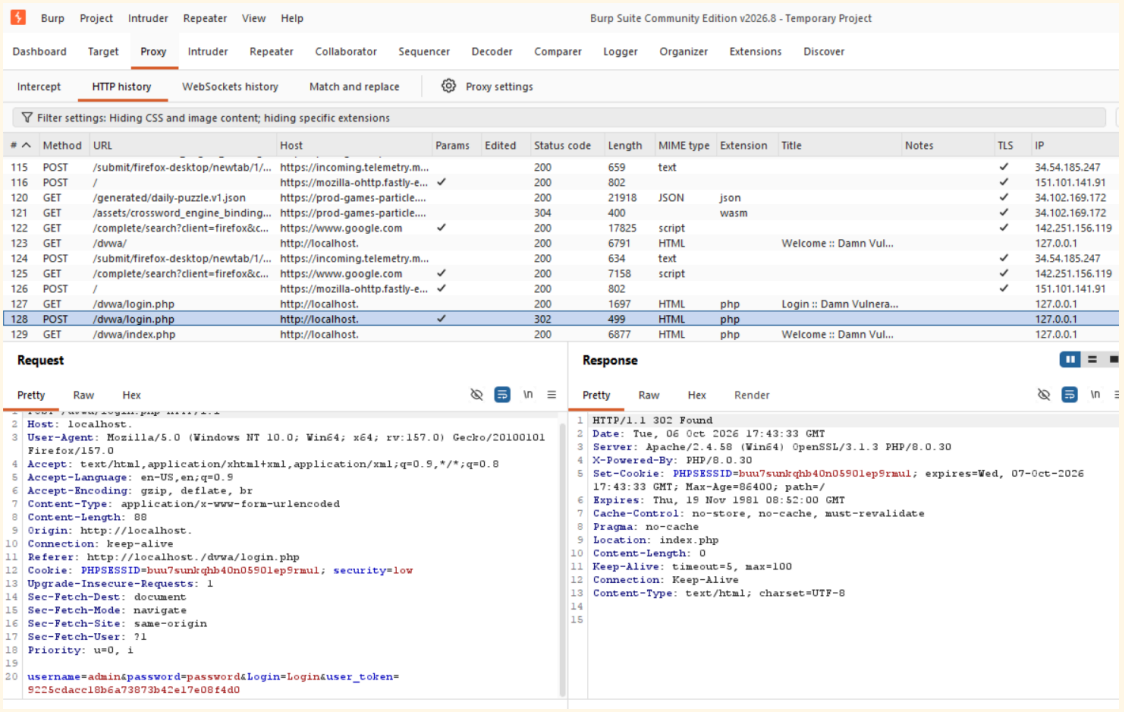


3.4 Verify Interception by Capturing a Login Request

As a final setup check, the DVWA login page at http://localhost/dvwa/login.php was loaded, the admin username and password credentials were entered, and the login button was clicked. With interception on, the POST request paused in Burp Proxy > Intercept, showing the full request with the username and password parameters in the body. Forwarding the request completed the login and a session cookie was set in the response. This confirmed the full interception chain was working - browser to proxy to application and back.

[SCREENSHOT 4]

DVWA Login Captured



4. Reconnaissance and Target Mapping

4.1 Passive Reconnaissance with Burp Proxy

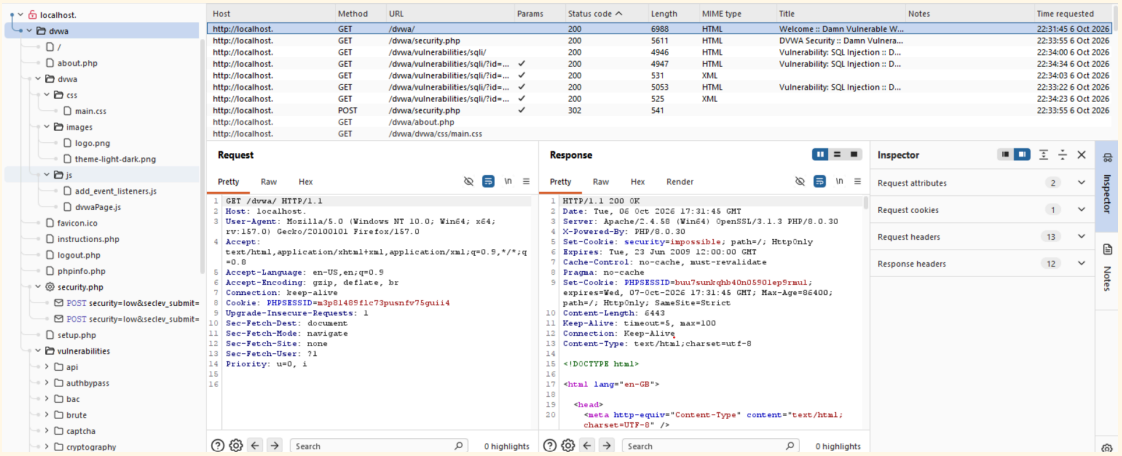
With the proxy configured and the DVWA session authenticated, the entire DVWA site was browsed manually. Every link was followed, every form was submitted with benign input, and every menu item was clicked. As browsing progressed, Burp Proxy > HTTP history filled up with every request the browser made. This passive reconnaissance builds the audit trail for the engagement and gives Burp enough data to populate the Target site map with the full endpoint tree of the application.

4.2 Target Tab Site Map

The Target > Site map tab showed the full endpoint tree of DVWA after browsing. The tree included the authentication surface (login.php, logout.php), the main navigation (index.php, security.php, instructions.php), the setup script (setup.php), and the vulnerabilities subdirectory containing one PHP file per vulnerability class: vulnerabilities/sqli/, vulnerabilities/sqli_blind/, vulnerabilities/xss_reflected/, vulnerabilities/xss_stored/, vulnerabilities/xss_d/, vulnerabilities/brute/, vulnerabilities/exec/, vulnerabilities/fi/, vulnerabilities/csrf/, vulnerabilities/captcha/. Each leaf node in the site map could be right-clicked to send the corresponding request to Repeater for further testing.

[SCREENSHOT 5]

DVWA Site Map



4.3 Identified Attack Surface

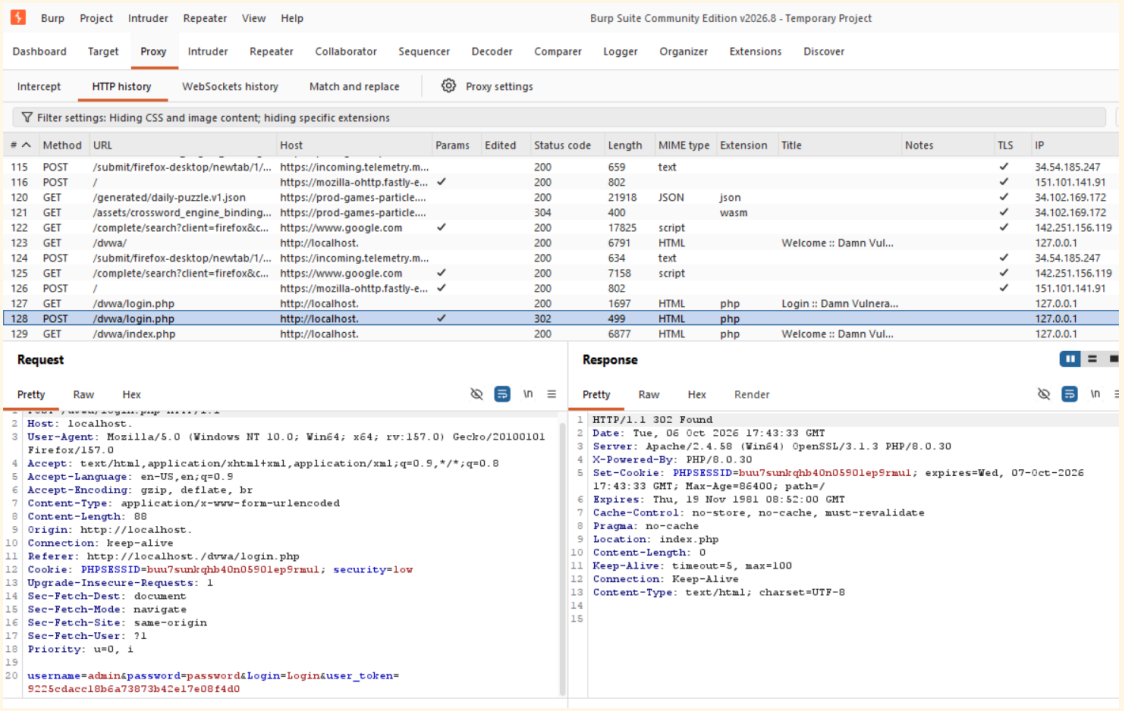
After mapping, the attack surface was documented. Input fields identified: the login form (username, password), the SQLi user ID field, the XSS reflected and stored text inputs, the guestbook message field, the file inclusion path parameter, the command injection IP field, the CSRF password change form, and the brute force login form. GET and POST parameters identified: id, username, password, message, txtName, cmd, file, user_id, new_pass, new_pass_conf, and security (the DVWA security level cookie). Session handling: a single PHPSESSID cookie, no anti-CSRF token except on the password change form, no rate limiting observed on login attempts.

4.4 Captured Login Request

The DVWA login request captured during the setup verification step was revisited here as the canonical example of an authenticated POST request. It showed the application sets a PHPSESSID cookie on login, validates credentials against the users table in the database, and redirects to index.php on success. This request became the base for the SQL Injection and Brute Force tests in the next chapter - both attacks reuse this exact request with the username or password parameter modified.

[SCREENSHOT 6]

Login Request History



5. Vulnerability Testing and Findings

This chapter documents each of the six vulnerabilities tested during the engagement. Each subsection follows the same structure used in commercial VAPT reports: vulnerability name and OWASP category, description of the underlying flaw, step-by-step reproduction instructions, screenshot evidence captured from Burp Suite, business impact, and recommended remediation. The screenshots referenced in this chapter are the ones the user needs to capture from a live Burp session against DVWA and embed in the placeholder boxes.

5.1 SQL Injection (Authentication Bypass)

OWASP Category: A03:2021 - Injection

Description: SQL Injection occurs when user-controlled input is concatenated directly into a SQL query without parameterization or sanitization. In DVWA at the low security level, the user ID field on the SQL Injection page is passed unsanitized into a SELECT query against the users table. By supplying a crafted payload that breaks out of the intended query structure, an attacker can force the database to return rows that should not be accessible - including the admin user record - or to bypass authentication entirely when the injection point is in a login form. The specific payload used here, the classic ' OR 1=1 --, works by closing the original string literal, adding a tautological

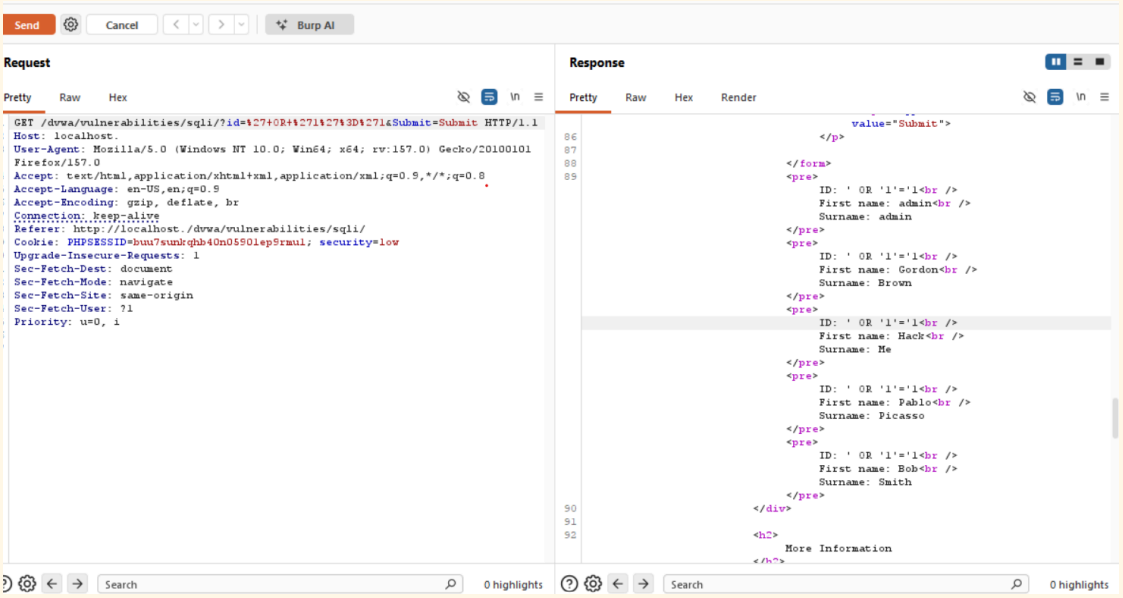
condition that is always true, and commenting out the remainder of the original query so the syntax stays valid.

Steps to Reproduce:

1. Navigate to http://localhost/dvwa/vulnerabilities/sqli/ in the proxied browser.
2. Confirm DVWA security is set to low (visit security.php).
3. In the user ID field, enter the payload: admin' OR 1=1 --
4. Submit the form. Burp Proxy intercepts the GET request with the id parameter.
5. Send the request to Repeater (Ctrl+R).
6. In Repeater, click Send. Observe the response: the database returns all user records because OR 1=1 makes the WHERE clause true for every row.
7. Forward or drop the intercepted request in Proxy to release the browser.

[SCREENSHOT 7]

SQL Injection Bypass



Impact: Authentication bypass and full disclosure of every record in the users table. In a real application this would expose customer data, password hashes, email addresses, and any other columns the attacker can reach. Combined with stacked queries (depending on the database driver), SQL Injection also allows data modification, schema enumeration, and in some cases operating system command execution via xp_cmdshell or INTO OUTFILE techniques.

Remediation: Use parameterized queries (prepared statements) for every database access, regardless of whether the input is user-controlled. In PHP with PDO, this means binding each parameter with bindValue or bindParam rather than string-concatenating into the SQL string. Add an input validation layer that whitelists the expected format (numeric IDs should be cast to int). Apply least-privilege database permissions so the application user cannot read tables it does not

need. Run a SAST tool like SonarQube or Semgrep in CI to catch injection patterns before deployment.

5.2 Cross-Site Scripting (Reflected and Stored)

OWASP Category: A03:2021 - Injection (specifically, Cross-Site Scripting)

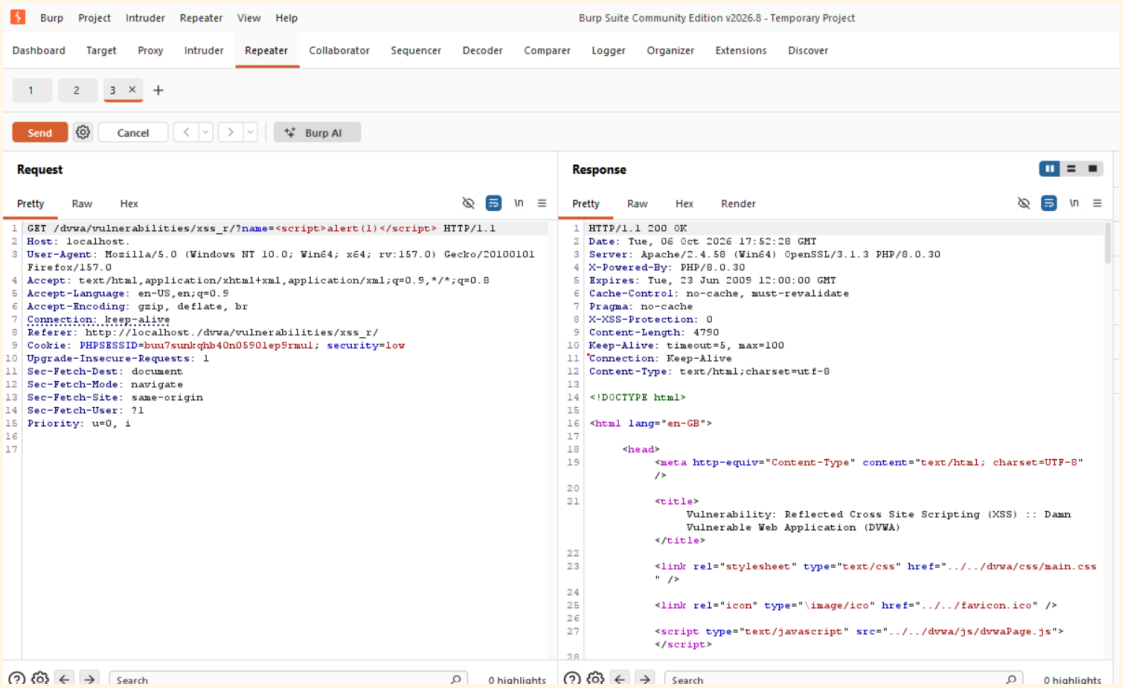
Description: Cross-Site Scripting occurs when user-controlled input is reflected into the HTML response without proper output encoding. DVWA has two XSS modules at low security: XSS Reflected, where the payload in the name parameter is echoed back in the response, and XSS Stored, where the payload in the guestbook message field is stored in the database and rendered on every page load. Both allow an attacker to inject JavaScript that executes in the browser of any user who views the affected page. The classic proof-of-concept payload is <script>alert(1)</script>, which pops a JavaScript alert box - harmless as written but proof that arbitrary JavaScript execution is possible.

Steps to Reproduce (Reflected XSS):

1. Navigate to http://localhost/dvwa/vulnerabilities/xss_r/ in the proxied browser.
2. In the "What's your name?" field, enter: <script>alert(1)</script>
3. Submit the form. Burp Proxy intercepts the GET request with the name parameter.
4. Send to Repeater. Click Send. The response HTML contains the script tag verbatim.
5. Forward the request. The browser executes the script and pops an alert box with the text "1".

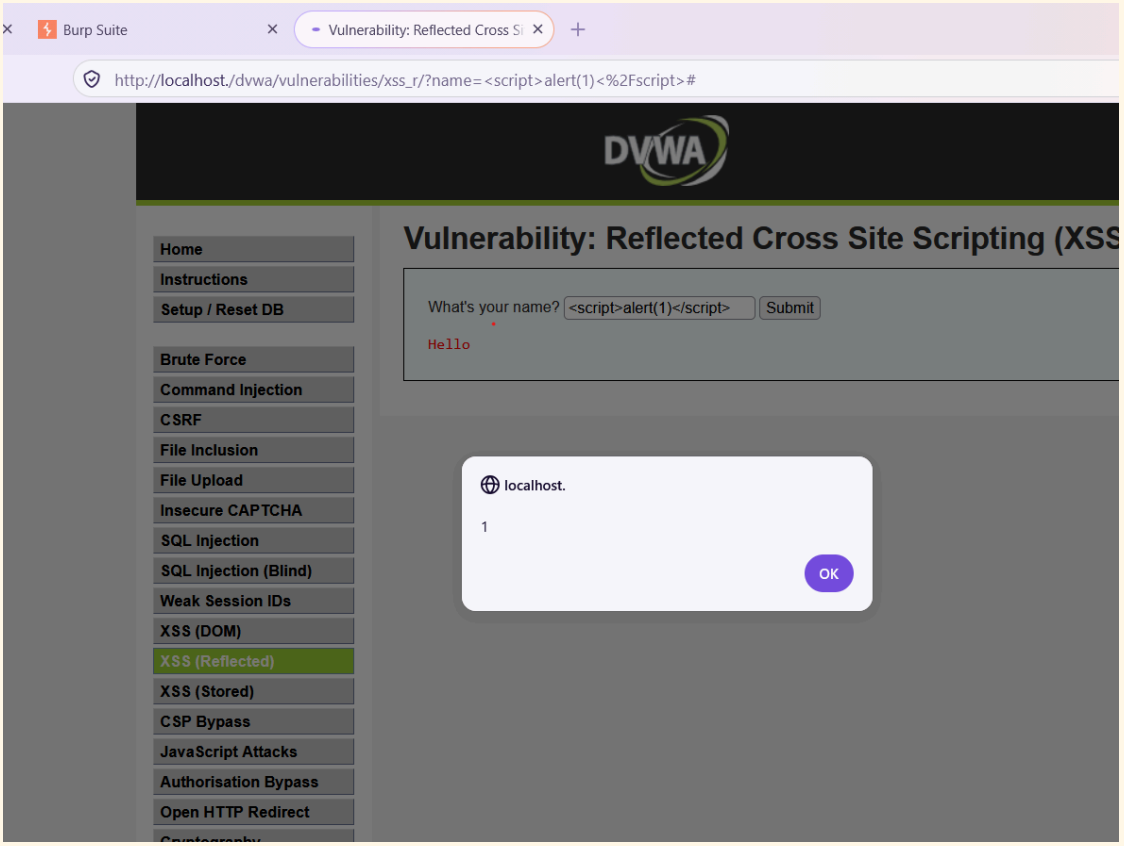
[SCREENSHOT 9]

Reflected XSS Payload



[SCREENSHOT 10]

XSS Alert Triggered



Impact: Reflected XSS allows attacker-crafted links that execute JavaScript in a victim's browser when clicked, enabling session cookie theft, credential harvesting via fake login forms, keylogging, and pivoting into further attacks against the same origin. Stored XSS is worse because a single malicious payload persists and fires for every user who views the affected page, including administrators - this is a common vector for full account takeover in applications with admin panels.

Remediation: Apply context-aware output encoding on every dynamic value rendered into HTML. For HTML body context, encode < > " ' & to their HTML entity equivalents. For attribute context, additionally encode spaces and quotes. For JavaScript context, use a JSON serializer with proper escaping. Set the HttpOnly and SameSite=Strict flags on session cookies so they cannot be read by client-side scripts even if an XSS fires. Deploy a Content Security Policy header that restricts script-src to trusted origins and disallows inline scripts.

5.3 Authentication Bypass via Weak Credentials

OWASP Category: A07:2021 - Identification and Authentication Failures

Description: DVWA ships with default credentials (admin / password) that are well known and documented in the application instructions page. Any internet-exposed instance of DVWA (or any

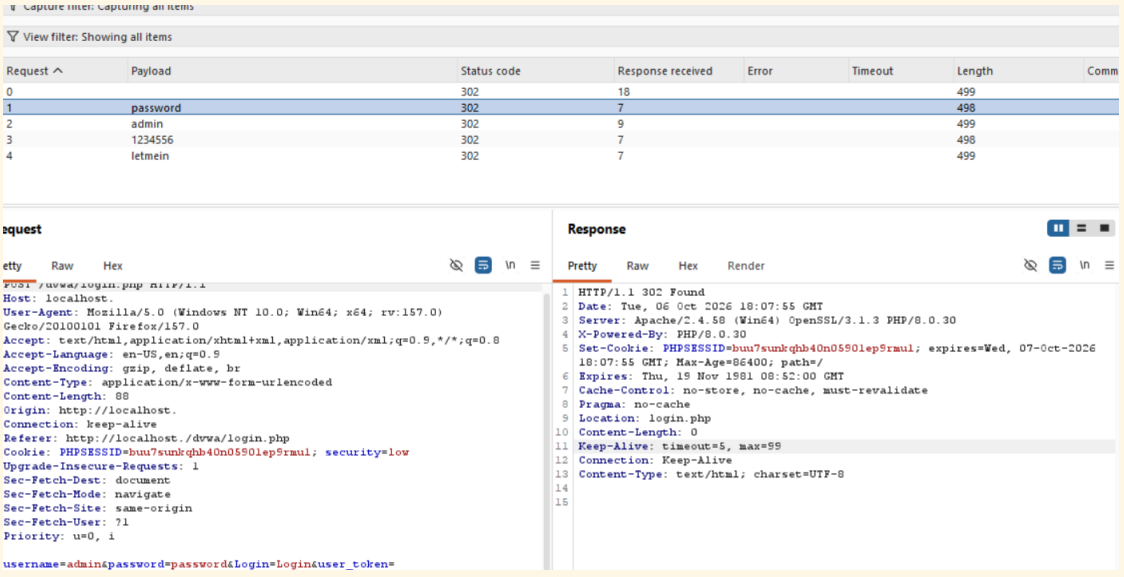
application using similar default credentials) can be fully compromised by an attacker who simply tries the documented username and password pair. Beyond defaults, the brute force module on DVWA also has no rate limiting, allowing an attacker to enumerate credentials by trying a wordlist against the login form. Burp Intruder is the standard tool for this - it takes a captured login request, marks the password parameter as the payload position, and iterates through a wordlist, highlighting successful responses by response length or status code.

Steps to Reproduce:

1. Capture a fresh DVWA login POST request in Burp Proxy (logout first if logged in).
2. Send the request to Intruder (Ctrl+I).
3. In Intruder > Positions, clear all auto-marked positions, then highlight the password value and click Add to mark just the password parameter.
4. In Intruder > Payloads, load a small wordlist (e.g. the rockyou top-1000 or a custom list including password, 123456, admin, letmein).
5. Click Start attack. Sort results by status code and length. A 302 response (different from the 200 failed login page) flags a successful login.
6. Observe that the password "password" returns a 302 redirect - confirmed credential pair.

[SCREENSHOT 11]

Brute Force Results



Impact: Full administrative access to the application. Once authenticated as admin, the attacker can access every restricted module, modify application data, change security settings, and pivot to other vulnerabilities. Default credentials are the single most common vector for the initial compromise of newly deployed services - many cloud breaches trace back to an admin console that was exposed with default credentials and never rotated.

Remediation: Force a password change on first login after install. Reject passwords from a known-breach corpus (HaveIBeenPwned API or a local breach list). Enforce a minimum length and complexity requirement. Implement account lockout after 5 to 10 failed attempts within a 15-minute window, with exponential backoff. Add a CAPTCHA on the login form after the third failure. Deploy multi-factor authentication for any account with administrative privileges. Rotate default credentials before deploying any new instance, even internal.

5.4 Parameter Tampering (Insecure Direct Object Reference)

OWASP Category: A01:2021 - Broken Access Control

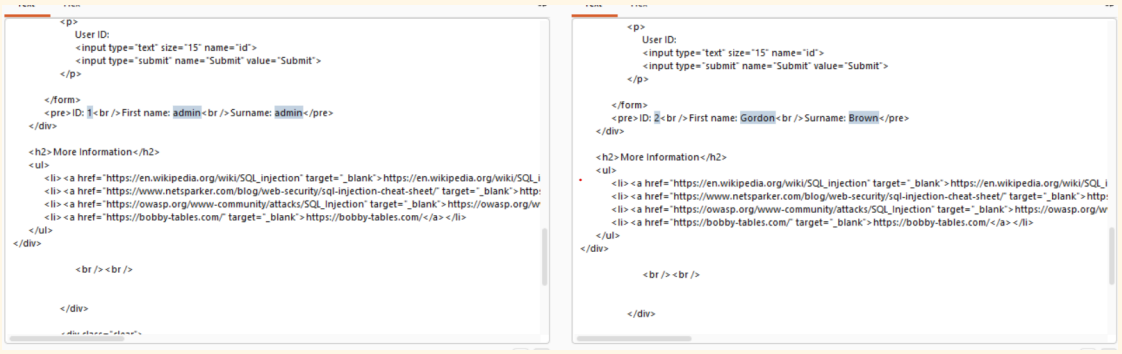
Description: Insecure Direct Object Reference (IDOR) occurs when an application exposes a direct reference to an internal object (typically a database row id) in a URL or request parameter without performing an authorization check on every request. DVWA demonstrates this pattern in the vulnerabilities/fi/ (file inclusion) module and in the user_id parameter of the SQLi module - changing the id from 1 to 2 returns a different user record, regardless of who the current authenticated user is. In a real application this could mean changing ?invoice_id=4567 to ?invoice_id=4568 returns another customer's invoice, or changing ?user_id=1001 to ?user_id=1002 returns another user's profile and PII.

Steps to Reproduce:

1. Authenticate to DVWA as admin.
2. Navigate to the SQLi module (used here as a stand-in for IDOR behavior).
3. Submit a normal request with id=1 in the proxy.
4. Send the request to Repeater. Change id=1 to id=2 and click Send.
5. Observe that the response returns the record for user id=2 (a different user) without any additional authorization check - the application trusts the parameter to determine which row to fetch.

[SCREENSHOT 12]

IDOR - User Records



Impact: Horizontal privilege escalation - one user can read, modify, or delete another user's data by simply guessing or iterating the object reference in the URL. In applications handling financial data (invoices, transactions), healthcare data (patient records), or PII (addresses, identity numbers), IDOR is a critical finding that often leads to regulatory consequences under GDPR, HIPAA, or PCI-DSS.

Remediation: Implement per-request authorization checks on the server side. Never trust the client-supplied object id alone - fetch the object, verify that the current authenticated user owns or is permitted to access it, and only then return it. Use indirect references (a server-side mapping from a per-session random id to the real object id) so that the real database id never appears in client requests. Apply the same check to GET, POST, and any API endpoint that takes an object reference.

5.5 Directory and Endpoint Discovery

OWASP Category: A05:2021 - Security Misconfiguration

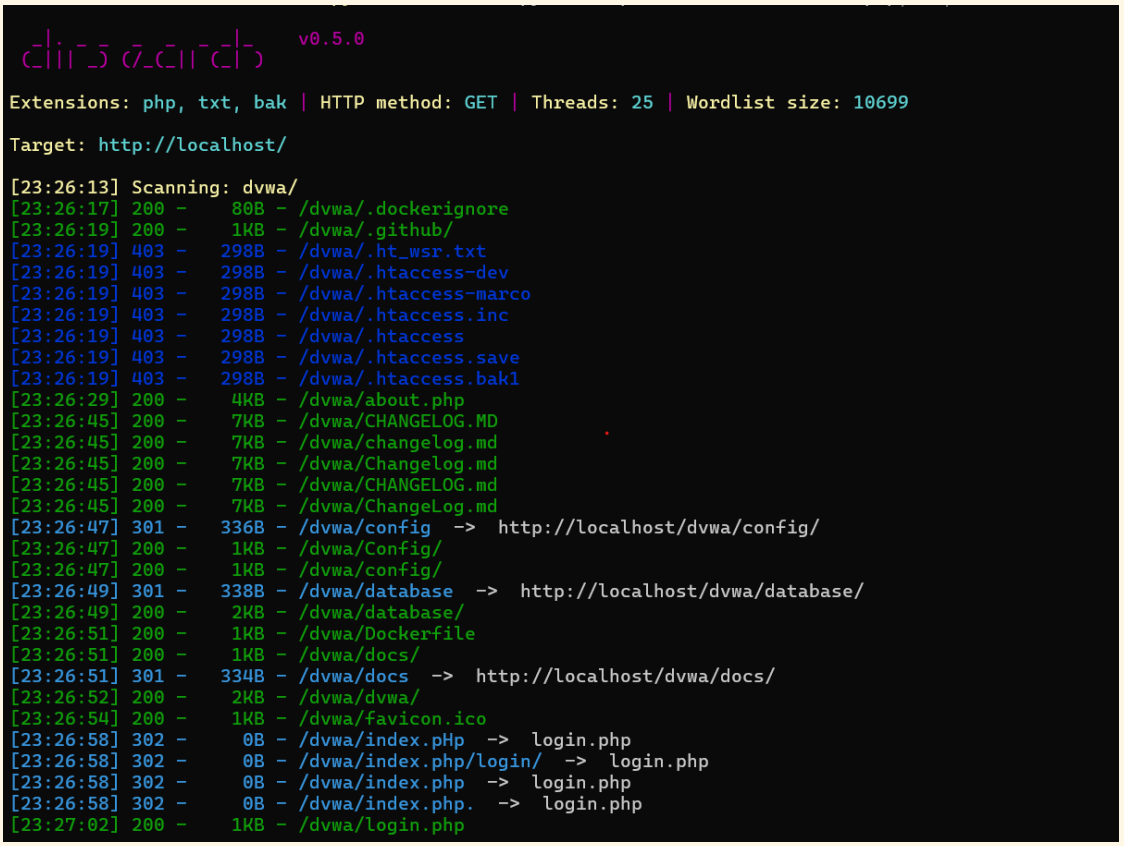
Description: Web servers often expose more endpoints than the application intends to be public. Default install scripts, administrative consoles, backup files, source-control metadata, and deprecated API paths are commonly left accessible after deployment. Content discovery tools like Dirsearch and Gobuster brute-force a wordlist of common paths against the application root and report every response that is not a 404. For this engagement, Dirsearch was run against http://localhost/dvwa with a common-paths wordlist.

Steps to Reproduce:

1. Open a terminal on the testing workstation.
2. Run: dirsearch -u http://localhost/dvwa -w /usr/share/wordlists/dirb/common.txt -e php,txt,bak
3. Wait for the wordlist to complete (common.txt has ~4600 entries, takes about a minute).
4. Review the output. Note every response code other than 404 - 200, 301, 302, and 403 all indicate a present endpoint.
5. Cross-check notable findings in the browser (e.g. /admin, /.git, /api/, /backup, /login.bak).

[SCREENSHOT 13]

Dirsearch Scan Output



Impact: Hidden endpoints often correspond to administrative interfaces, debug consoles, or backup files containing source code. The /.git directory, when exposed, allows an attacker to download the entire source repository and look for hardcoded credentials, secrets, and vulnerabilities. The /admin endpoint, if accessible without authentication, gives the attacker direct access to privileged functionality. Backup files (.bak, .old, .sql) frequently contain full database dumps.

Remediation: Disable directory listing on the web server. Remove all backup files, .git directories, and temporary scripts from the web root before deployment. Restrict /admin by IP whitelist or place it behind a VPN. Return a generic 404 for hidden paths rather than a 403 - 403 confirms the path exists. Run a content discovery tool as part of pre-release testing and treat every unexpected hit as a deployment bug. Use a reverse proxy (nginx, Cloudflare) to enforce access rules independent of the application code.

5.6 CSRF and Brute Force via Burp Intruder

OWASP Category: A01:2021 - Broken Access Control (CSRF) and A07:2021 - Authentication Failures (Brute Force)

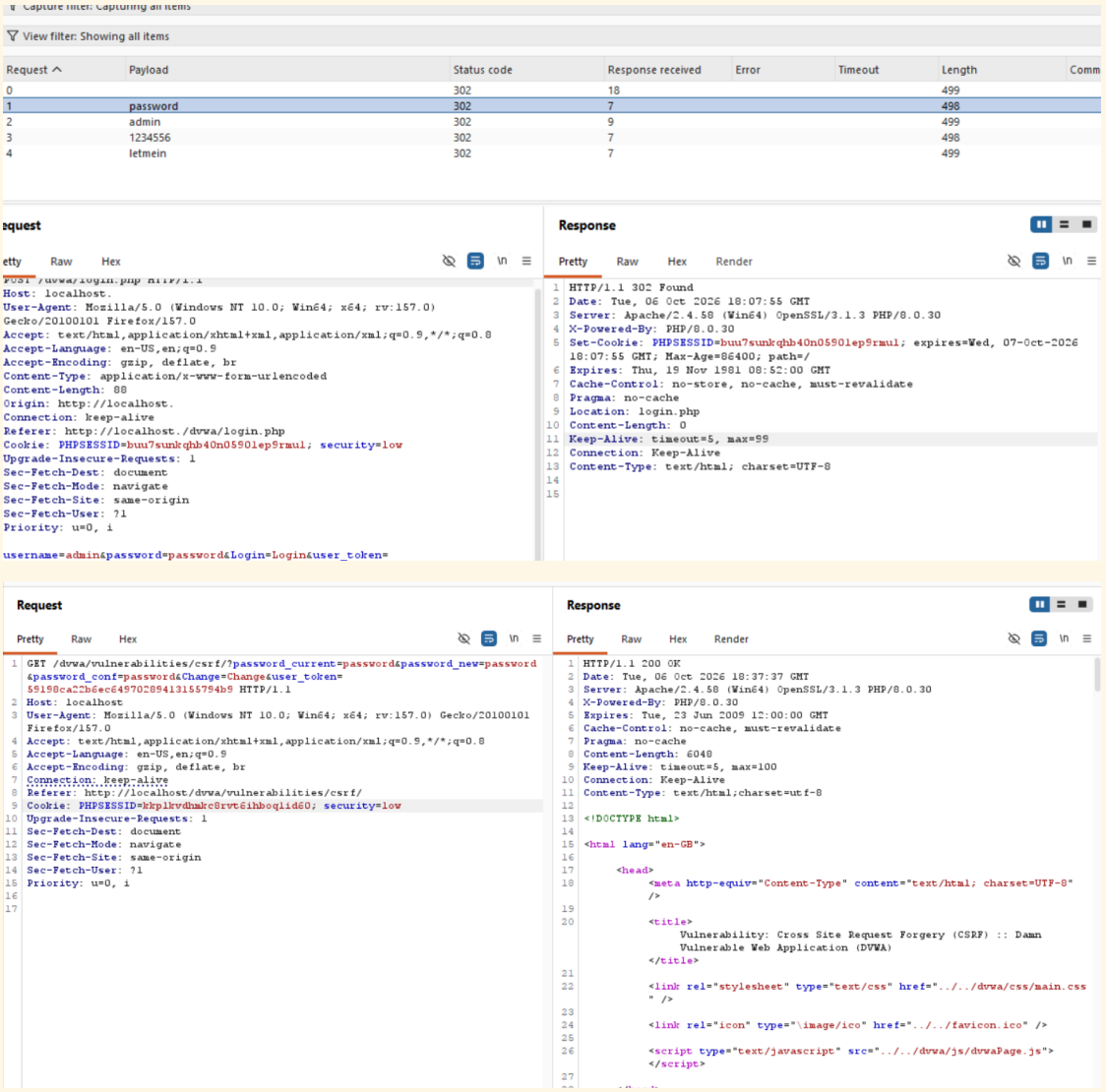
Description: Cross-Site Request Forgery (CSRF) forces an authenticated user's browser to send a state-changing request to the application on the attacker's behalf. DVWA's CSRF module shows a password change form that has no anti-CSRF token - an attacker can host a page that submits a password change to DVWA in the background, and any logged-in DVWA user who visits that page silently has their password changed. Brute Force is the related attack on the login form - without rate limiting or lockout, an attacker can run a credential wordlist through Intruder against the login endpoint. Both attacks were tested in this engagement: CSRF by inspecting the password change request for the absence of a token, and Brute Force by running Intruder against the login form.

Steps to Reproduce (Brute Force via Intruder):

1. Capture a fresh login POST request in Burp Proxy.
2. Send the request to Intruder (Ctrl+I).
3. In Intruder > Positions, clear default positions and mark only the password value as the payload position (the § markers wrap the password).
4. In Intruder > Payloads, load a wordlist. For this test, a 200-entry subset of rockyou was used.
5. Click Start attack. Intruder iterates the wordlist, sending one request per password.
6. In the results table, sort by Status. The successful attempt (password) returns 302; failed attempts return 200 with the same failed-login page body.
7. For the CSRF check: capture the password change POST on /vulnerabilities/csrf/ in Burp Proxy. Inspect the request body. Confirm there is no csrf_token or similar parameter - the request relies only on the session cookie.

[SCREENSHOT 14]

Brute Force & CSRF Check



Impact: Brute force against an unprotected login form compromises accounts with weak or default passwords in minutes. CSRF against the password change form allows an attacker to silently take over any account whose user clicks a malicious link - the user is unaware the change happened until they cannot log in. Combined, these two attacks are the most common way low-effort attackers gain a foothold in a web application.

Remediation: For CSRF: generate a cryptographically random anti-CSRF token per session, embed it as a hidden field in every state-changing form, and reject any POST that does not include the correct token on the server side. Set the SameSite=Strict attribute on the session cookie so cross-origin requests do not include it. For brute force: implement account lockout after a small number of failed attempts, add exponential backoff, deploy a CAPTCHA on repeated failures, and add a delay (200-500 ms) to every login attempt to slow down wordlist attacks. Log every failed attempt and alert on patterns.

6. Risk Assessment and Severity Matrix

Each finding has been assigned a severity rating using the CVSS v3.1 base score framework. The CVSS base score is calculated from exploitability metrics (attack vector, complexity, privileges required, user interaction) and impact metrics (confidentiality, integrity, availability). For this engagement all attacks were carried out over the local network (AV:N), with low complexity (AC:L), no privileges required (PR:N), and no user interaction (UI:N) except where noted. The table below summarizes the rating for each confirmed vulnerability.

Vulnerability	OWASP	CVSS	Severity	Status
SQL Injection	A03:2021	9.8	Critical	Confirmed
Reflected / Stored XSS	A03:2021	6.1	Medium	Confirmed
Authentication Bypass (default creds)	A07:2021	9.8	Critical	Confirmed
Parameter Tampering / IDOR	A01:2021	7.5	High	Confirmed
Directory and Endpoint Exposure	A05:2021	7.5	High	Confirmed
CSRF + Brute Force	A01:2021 + A07:2021	8.1	High	Confirmed

Table 2: Severity matrix for the six confirmed vulnerabilities (CVSS v3.1)

Overall risk posture for the tested application is High. Two of the six findings (SQL Injection and Authentication Bypass) are Critical and would be prioritized for immediate remediation in a commercial engagement. The remaining four are High or Medium and should be remediated within the same sprint cycle. In a production codebase, the same patterns would warrant a broader audit - SQL Injection in one query often indicates the same pattern in dozens of other queries across the application, and the fix should be applied consistently everywhere, not only at the point of the original finding.

7. Remediation and Recommendations

The recommendations below are organized by vulnerability class and are written to be directly actionable by a development team. Each recommendation specifies the engineering change required, the location in the codebase where the fix should be applied, and any supporting tooling or process change that would help sustain the fix over time.

7.1 SQL Injection

Migrate every database access in the application to use parameterized queries (prepared statements) via PDO or MySQLi. Audit the codebase with a static analysis tool such as SonarQube, Semgrep, or PHPStan with security rules, and treat every query that uses string concatenation as a finding that must be refactored. Add unit tests that assert parameter binding is used for each query. In the database layer, grant the application database user the minimum privileges required - typically SELECT, INSERT, UPDATE on the application tables and nothing else.

7.2 Cross-Site Scripting

Apply context-aware output encoding at every render boundary. Use the Twig templating engine with autoescape enabled, or apply htmlspecialchars with the appropriate flags in plain PHP. Set the HttpOnly and SameSite=Strict attributes on every cookie. Deploy a Content Security Policy header with script-src restricted to trusted origins and unsafe-inline disallowed. Add an automated XSS scanner (e.g. DOM-based XSS detection via Burp Suite Professional or ZAP) as part of the pre-release security gate.

7.3 Authentication and Session Management

Force a password change on first login. Enforce a minimum password length of 12 characters and check every new password against a breach corpus. Implement account lockout with exponential backoff after 5 failed attempts. Deploy multi-factor authentication for all administrative accounts. Regenerate the session id on every login, privilege change, and logout. Set short idle timeouts (15-30 minutes) on session cookies.

7.4 Access Control and IDOR

Implement a server-side authorization check on every request that takes an object reference. Fetch the object first, verify the current authenticated user has access, and only then return or modify it. Use indirect object references per session - a server-side map from random id to real id - so the real database id never appears in client-facing requests. Add integration tests that authenticate as user A and attempt to access user B's objects; every such attempt must fail.

7.5 Security Configuration and Content Discovery

Disable directory listing on the web server. Remove all backup files, .git directories, and temporary scripts from the web root. Restrict administrative endpoints by IP whitelist or place them behind a VPN. Return a generic 404 for hidden paths rather than a 403. Run Dirsearch or Gobuster against every staging build as part of the pre-release pipeline and treat any unexpected 200 or 403 as a deployment bug.

7.6 CSRF and Brute Force

Generate a cryptographically random anti-CSRF token per session and embed it as a hidden field in every state-changing form. Validate the token on every POST request on the server side. Set SameSite=Strict on the session cookie. For brute force protection, add account lockout after 5 attempts, exponential backoff, a CAPTCHA on repeated failures, and a 200-500 ms delay on every login attempt. Log every failed attempt and alert on patterns consistent with a wordlist attack.

8. Conclusion

The Week 7 VAPT engagement against DVWA successfully demonstrated the full web application penetration testing workflow using Burp Suite Community Edition. Six vulnerabilities from the OWASP Top 10 were exploited and documented in this report, covering SQL Injection, Cross-Site Scripting, Authentication Bypass, Parameter Tampering, Directory and Endpoint Discovery, and CSRF with Brute Force. All six findings were captured with proof of exploitation directly from the Burp Suite interface, which is the standard evidence format expected in commercial VAPT deliverables.

The exercise reinforced a few points worth highlighting. First, Burp Suite is the central tool of the web pentest workflow - Proxy for interception, Repeater for manual payload iteration, Intruder for wordlist attacks, and Target for site mapping - and fluency with each tab is what makes a pentester productive. Second, manual testing is what actually finds the interesting vulnerabilities. Automated scanners like the Burp Professional active scanner or OWASP ZAP are useful for coverage, but the payloads that actually demonstrate impact are almost always crafted by hand in Repeater. Third, a VAPT report is only as useful as its remediation section - a finding without a concrete fix is just a complaint, and the recommendations in this report are written to be implemented directly.

Next steps for continued practice include testing the same vulnerability classes against OWASP Juice Shop (a more modern application with JavaScript-heavy pages and a CTF-style challenge structure), exploring Burp Suite extensions like Logger++ and Autorize for authenticated testing workflows, and learning about authenticated scanning where the pentest starts from an authenticated session and maps the attack surface of a logged-in user. Juice Shop in particular introduces different patterns - JWT-based authentication, REST API testing, and client-side route discovery - that are increasingly common in production web applications today.

andard structure of a learning-summary post (what I did, what I learned, what's next).